Лабораторная работа №2

Проблема собственных значений матрицы

Цель работы

Исследовать методы вычисления сингулярного разложения на примере обработки изображений, сравнить эффективность степенного метода и метода вращений Якоби, изучить применение усеченного SVD для сжатия данных.

Краткие теоретические сведения

Описание степенного метода

A — диагонализуема

x_0 — произвольный вектор с ненулевой проекцией на v_1 (вектор, соответствующий спектральному радиусу).

$$x_{k+1} = \frac{Ax_k}{\|Ax_k\|}$$

(x_k) сходится по направлению v_1.

Скорость сходимости определяется соотношением $\left|\frac{\lambda_2}{\lambda_1}\right|$. Чем ближе к 0, тем быстрее сходится к v_1.

$\|x_{k+1} - x_k\| < \varepsilon$ - один из критериев остановки.

Получаем аппроксимацию собственного вектора $\Rightarrow$ можем найти собственное значение

- λ_1 — не может быть кратным корнем
- λ_1 — не комплексный корень

Описание сингулярного разложения

$A \in \mathrm{Mat}_{\mathbb{R}}(m, n)$, $\mathrm{rank}(A) = r \leqslant \min(m, n)$

Опр. Сингулярным разложением называется представление $A = U\Sigma V^T$, где $U_{m\times m}$, $V_{n\times n}$ — ортогональные, $\Sigma \in \mathrm{Mat}(m, n)$ с сингулярными значениями $\{\sigma_1 \geqslant \sigma_2 \geqslant \ldots \geqslant \sigma_r\}$

Опр. $\sigma \geqslant 0$, $\sigma \in \mathbb{R}$ называется сингулярным числом матрицы M, если $\exists$ 2 единичных вектора $u \in \mathbb{R}^m$, $v \in \mathbb{R}^n$:

$$Mv = \sigma u$$
$$M^\dagger u = \sigma v$$

Векторы u, v называются левым и правым сингулярными векторами.

Алгоритм построения разложения:

1. $A^T A \in \mathbb{R}^{n\times n}$ — симметричная полож. опр.
2. Для $A^T A$ ищем с.з. и с.в. $\lambda_i \geqslant 0$

$$A^T A v_i = \lambda_i v_i, \qquad \|v_i\| = 1$$

$\exists$ собств. базис (оператор скалярного типа, собств. базис ортогонален)

3. $\sigma_i = \sqrt{\lambda_i}$, из $\{\sigma_i\}$ выбираем r наибольших.

   v_i — столбцы матрицы V

4. $\forall \sigma_i$ вычисляем левый сингулярный вектор

$$u_i = \frac{1}{\sigma_i} A v_i \quad (r \text{ штук})$$

5. u_i — первые r столбцов U, остальные столбцы выбираем так, чтобы в итоге получить ортог. матрицу (до ортонормированного базиса).

Описание метода вращений Якоби

Работает для действ. симметричных матриц. $A \in \mathbb{R}^{n \times n},\ A \in \mathrm{Sym}(n)$.

Утв. Для матрицы A, удовл. таким усл., $\exists$ ортогональное преобразование U, такое что $U^T A U = \mathrm{diag}(\lambda_1, \dots, \lambda_n)$.

$$U_{kl} = \begin{pmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & \cos\varphi & \dots & -\sin\varphi & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & \sin\varphi & \dots & \cos\varphi & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{pmatrix} \quad \text{вращение } O_{kl}$$

$$\tilde{A} = U_{kl}^T A U_{kl}, \quad A_{k+1} = U_k^T A_k U_k$$

Идея метода: последовательное уменьшение внедиагональных элементов с помощью серии элементарных вращений.

Хотим исключить (обнулить) эл-т $a_{lk} = a_{kl}$.

$$(\tilde{A})_{kl} = (\tilde{A})_{lk} = 0$$

Если $a_{kk} - a_{ll} = 0 \implies \varphi = \frac{\pi}{4}$

Если $a_{kk} - a_{ll} \neq 0 \implies \mathrm{tg}(2\varphi) = \frac{2a_{kl}}{a_{kk} - a_{ll}}, \quad |\varphi| < \frac{\pi}{4}$

При стремлении недиагональных элементов к нулю матрица A_k сходится к диагональной, при этом на диагонали стоят собственные значения.

Применение SVD для обработки изображений

- Сжатие изображений
- Удаление шума
- Выделения основных признаков изображения для задач машинного обучения

Ход работы

Описание исходного изображения

Разрешение: 640×853

Изображение вечернего пляжа.



Рис. 1: Исходное изображение

Листинг кода с комментариями

Реализация основных функций

```
def power_method(matrix, iterations):
    rng = np.random.default_rng(42)
    x_k = rng.standard_normal(matrix.shape[1])
    lambda_k = 0

    for _ in range(iterations):
        dot = matrix @ x_k
        x_k_plus_1 = (dot) / np.linalg.norm(dot)

        # Вычисление отношения Релея
        lambda_new = ((x_k_plus_1.T @ matrix @ x_k_plus_1) /
                      (x_k_plus_1.T @ x_k_plus_1))

        x_k = x_k_plus_1
        lambda_k = lambda_new

    return lambda_k, x_k
```

Listing 1: Степенной метод

```python
def svd(matrix, iterations, k=0):
    if k == 0:
        r = np.linalg.matrix_rank(matrix)
    else:
        r = k

    A = matrix.copy().astype(float)

    m = A.shape[0]
    n = A.shape[1]
    sigma = np.zeros(A.shape)
    V = np.zeros((n, n))
    U = np.zeros((m, m))

    for i in range(r):
        B = A.T @ A
        lambd, v = power_method(B, iterations=iterations)
        sigma_i = np.linalg.norm(A @ v)
        u = 1 / sigma_i * (A @ v)

        sigma[i, i] = sigma_i
        V[:, i] = v
        U[:, i] = u

        A = A - sigma_i * np.outer(u, v)

    return sigma, U, V
```

Listing 2: Реализация SVD

```python
def jacobi_rotation_method(matrix, tol, max_iter=10000):
    A = matrix.copy().astype(float)
    n = A.shape[0]
    eig_vectors = np.eye(n)
    diag_indices = np.arange(n)

    for _ in range(max_iter):
        abs_A = np.abs(A)
        abs_A[diag_indices, diag_indices] = 0.0
        flat_index = np.argmax(abs_A)
        row, col = np.unravel_index(flat_index, A.shape)

        if abs_A[row, col] < tol:
            break

        app = A[row, row]
        aqq = A[col, col]
        apq = A[row, col]

        if abs(app - aqq) < 1e-15:
            phi = np.pi / 4
        else:
            phi = 0.5 * np.arctan2(2 * apq, app - aqq)

        c = np.cos(phi)
        s = np.sin(phi)

        row_A = A[row, :].copy()
        col_A = A[col, :].copy()

        A[row, :] = c * row_A - s * col_A
        A[col, :] = s * row_A + c * col_A

        A[:, row] = A[row, :]
        A[:, col] = A[col, :]

        c2 = c * c
        s2 = s * s
        A[row, row] = c2 * app - 2 * c * s * apq + s2 * aqq
        A[col, col] = s2 * app + 2 * c * s * apq + c2 * aqq

        A[row, col] = 0.0
        A[col, row] = 0.0

        ev_row = eig_vectors[:, row].copy()
        ev_col = eig_vectors[:, col].copy()
        eig_vectors[:, row] = c * ev_row - s * ev_col
        eig_vectors[:, col] = s * ev_row + c * ev_col

    return A, eig_vectors
```

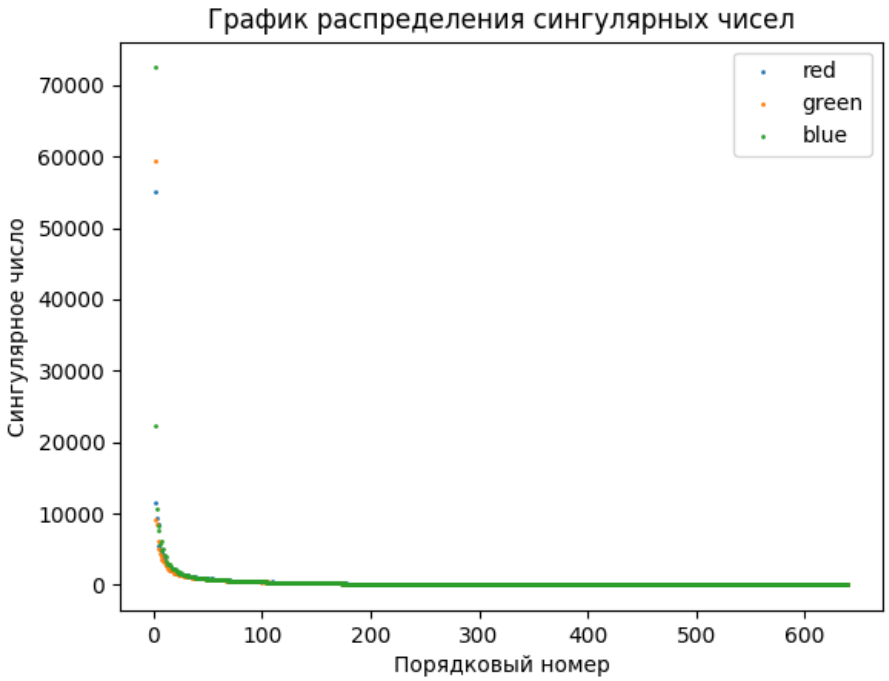

Listing 3: Метод вращений Якоби

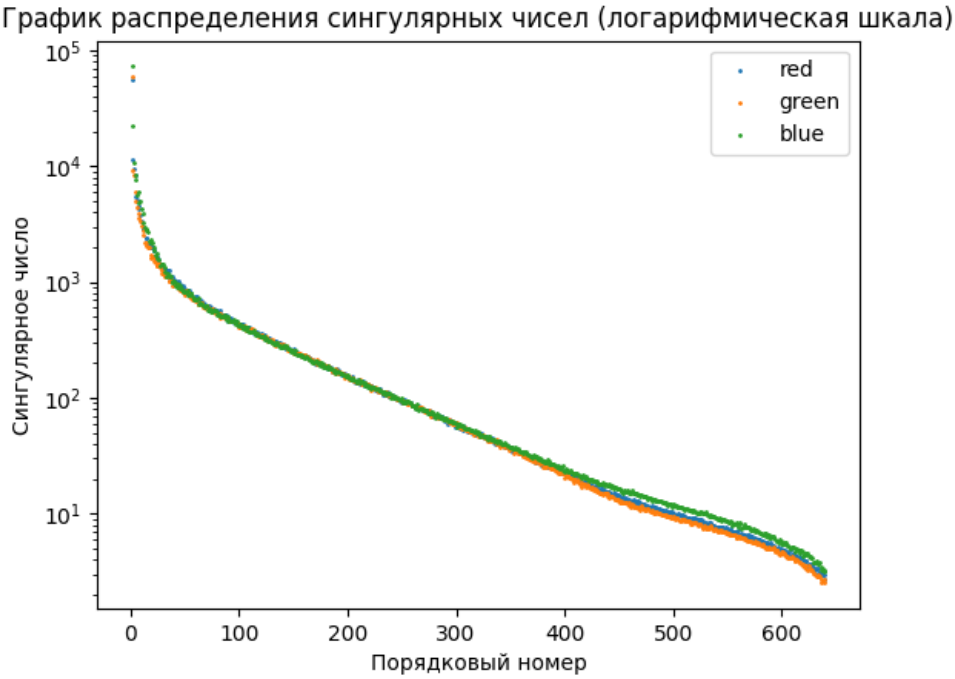
Полный notebook с графиками

Таблица результатов для разного числа итераций

Количество итераций \ Каналы	**Red**	**Green**	**Blue**
10	$4.551 \cdot 10^{-11}$	$4.801 \cdot 10^{-11}$	$6.022 \cdot 10^{-11}$
50	$4.560 \cdot 10^{-11}$	$4.817 \cdot 10^{-11}$	$6.230 \cdot 10^{-11}$
100	$4.563 \cdot 10^{-11}$	$4.864 \cdot 10^{-11}$	$6.229 \cdot 10^{-11}$

Графики распределения сингулярных чисел





Визуализация восстановленных изображений при разных k

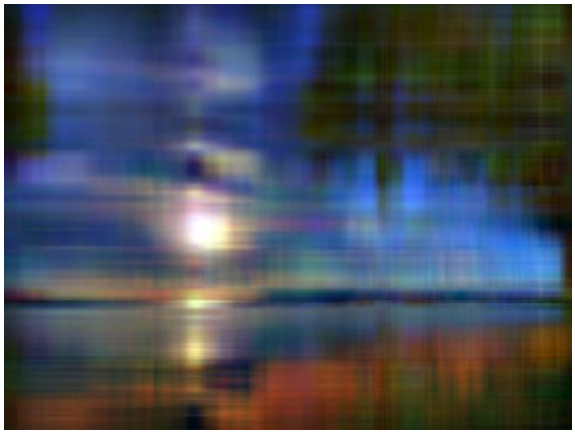


Рис. 2: Восстановленное изображение при $k = 5$

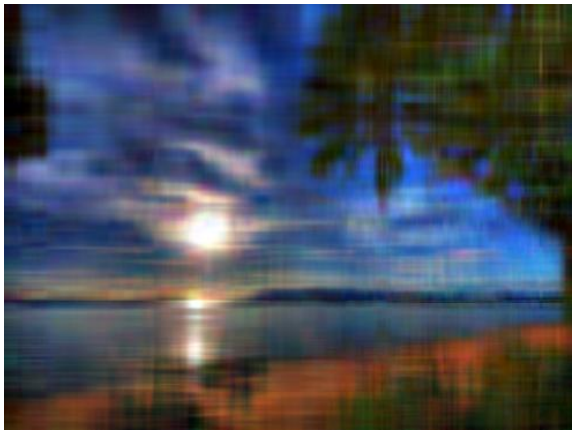


Рис. 3: Восстановленное изображение при $k = 10$



Рис. 4: Восстановленное изображение при $k = 20$



Рис. 5: Восстановленное изображение при $k = 50$

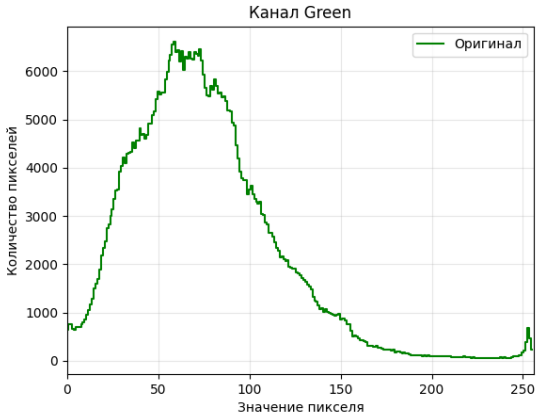


Рис. 8: Распределение значений пикселей для зеленого канала



Рис. 6: Восстановленное изображение при $k = 100$

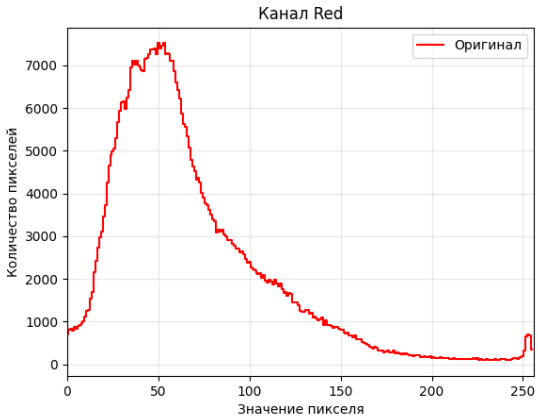


Рис. 7: Распределение значений пикселей для красного канала

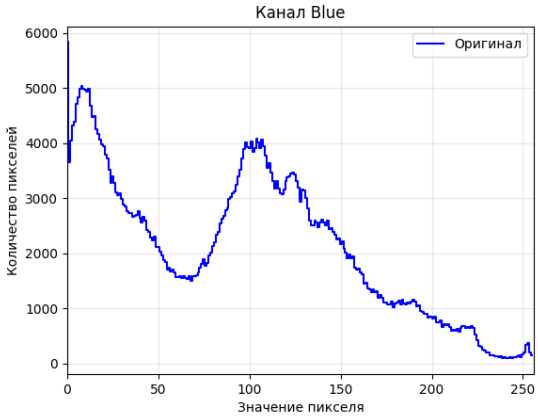


Рис. 9: Распределение значений пикселей для синего канала

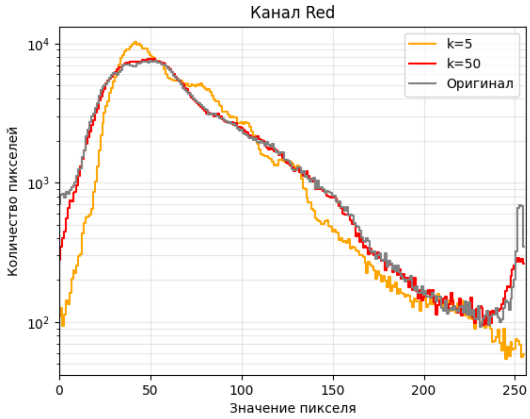


Рис. 10: Распределение яркости в красном канале (логарифмический масштаб)

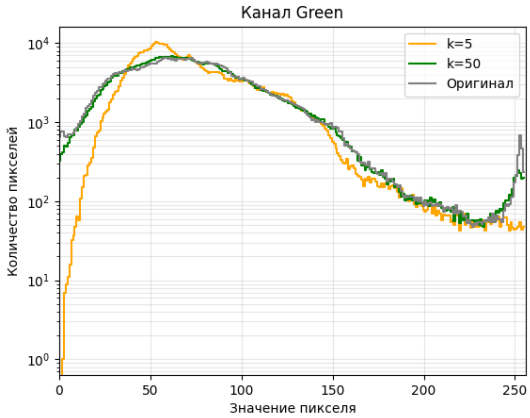


Рис. 11: Распределение яркости в зеленом канале (логарифмический масштаб)



Рис. 14: Изображение с щумоподавлением, k=100

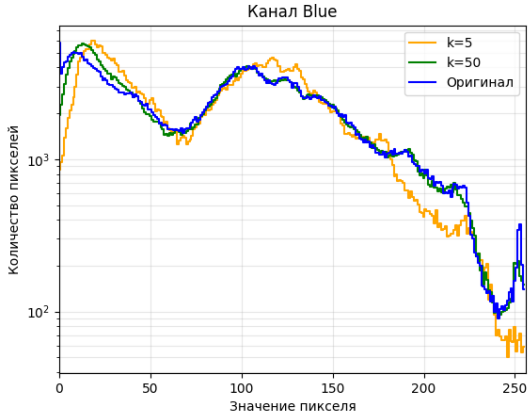


Рис. 12: Распределение яркости в синем канале (логарифмический масштаб)

Использование усеченного SVD для шумоподавления



Рис. 13: Изображение с гауссовским шумом

Анализ результатов

1. **Как число итераций степенного метода влияет на точность разложения?**

   Исходя из таблицы результатов для разного числа итераций, число итераций степенного метода практически не влияет на точность разложения. С увеличением числа итераций до 50-100 норма разности исходной матрицы и восстановленной увеличивается в $10^{-12} - 10^{-13}$.

2. **Какое минимальное число сингулярных чисел обеспечивает приемлемое качество изображения?**

   При 50 сингулярных числах качество изображения приемлемое.

3. **Какой метод (степенной или Якоби) оказался более эффективным по числу операций?**

   Степенной метод более эффективный по сравнению с методом Якоби.

4. **Есть ли различия в распределении сингулярных чисел для разных цветовых каналов?**

   Форма распределения для всех цветовых каналов имеет один вид, но значения максимальных сингулярных значений различаются (≈ 53000 для красного цветового канала, ≈ 60000 для зеленого цвета, ≈ 71000 для синего канала).

5. **Какова степень сжатия данных при использовании усеченного SVD?**

   Степень сжатия при использовании усеченного сингулярного разложения вычисляется по формуле $\frac{M \cdot N}{k(M+N+1)}$, где M и N - размеры матрицы, а k - количество сингулярных значений.

Выводы о применимости усеченного SVD для сжатия изображений

1. Усеченное сингулярное разложение позволяет значительно уменьшить объем данных при сохранении приемлемого качества изображения. При использовании $k = 50$ сингулярных чисел из исходных 640 степень сжатия равна 7.

2. Качество восстановленного изображения зависит от количества сохраняемых сингулярных чисел k. При малых значениях k (5-10) изображение сильно искажено. При $k \geq 50$ качество становится приемлемым, а при $k = 100$ различия с оригиналом практически не заметны.

Выводы о проделанной работе

В ходе выполнения лабораторной работы были исследованы методы вычисления сингулярного разложения на примере обработки изображений, было проведено сравнение эффективности степенного метода и метода вращений Якоби, изучено применение усеченного SVD для сжатия данных.